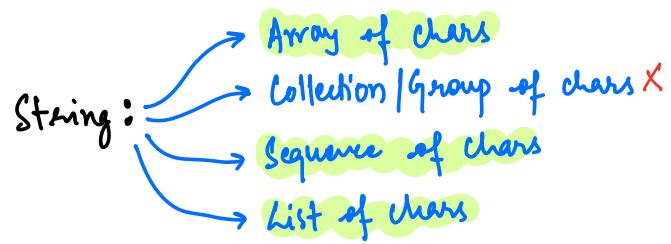


## Today's content:-

- Intro
- flip
- Sort int?
- Reverse substring
- longest palindromic substring



"a b c d"  
"b a c d" > order matters for strings

ASCII

|          |           |          |
|----------|-----------|----------|
| 'A' → 65 | 'a' → 97  | '0' → 48 |
| 'B' → 66 | 'b' → 98  | '1' → 49 |
| 'C' → 67 | 'c' → 99  | ⋮        |
| 'D' → 68 | 'd' → 100 | ⋮        |
| ⋮        | ⋮         | ⋮        |
| 'Z' → 90 | 'z' → 122 | '9' → 57 |

Diagram: A pink arrow labeled 32 points from the circled value 65 to the circled value 97. Another pink arrow labeled 32 points from the circled value 68 to the circled value 100.

Q1) Given a `char[]`, toggle every character  
 ↳ Capital ↔ Small

Note:- I/p contains only lower case and upper case english alphabets  
 Can't use any in-built functions.

e.g. `aBcAEd` → `AbCaEd`

```

toggle(char[] s) {
    for (int i=0; i < s.size(); i++) {
        // if s[i] is lowercase
        if (s[i] >= 97 && s[i] <= 122) {
            s[i] = s[i] - 32;
        }
        else {
            s[i] = s[i] + 32;
        }
    }
}
    
```

$T.C = O(N)$   
 $S.C = O(1)$

$s[i] = s[i] \wedge 32;$   
 ↳  $2^5$   
 ↳  $1 < 5$

Observations:-

1) Diff here is always 32

2) Binary of  $32 = 2^5 = 100000$

$'Z' = 90 = 1001100$   
 $+32 \rightarrow$   
 $'z' = 122 = 1111100$

Always set ←

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|

↳ Always unset for v.c characters.

Sort given ch[7], in alphabetical order.

S.C =  $O(1)$  (We can't use extra space)

e.g.  $s = d a b a c d b$

o/p = a a b b c d d

constraints:-

$$1 \leq N \leq 10^5$$

'a'  $\leq$  ch[i]  $\leq$  'z'

# Use inbuilt sort() function.  $\rightarrow T.C = O(N \log N)$

### Optimisation:-

2) We only have lower case characters

$(a-z) = 26 \text{ characters.}$

(97-122)

2)

|   |   |   |
|---|---|---|
| d | → | 2 |
| a | → | 2 |
| c | → | 1 |
| b | → | 2 |

iterate from  $(a' \rightarrow z')$

$a a b b c d d \rightarrow \text{Ans}$

3) Take an arr[] of size 26

int[1 count[26] =

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | - | - | - | - |
| 0 | 0 | 0 | 0 |   |   |   |   |

...

|    |    |    |
|----|----|----|
| 23 | 24 | 25 |
| 0  | 0  | 0  |

'ä' → 'z'

↓ ↓

97 125

$q = 97$

 $b = 98$ 
$$b = 10$$

CE 99

i

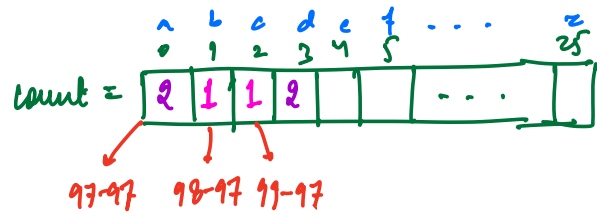
$$Z = 122$$

```

for (i=0; i<N; i++) {
    // index of s[i] in count[]
    int index = s[i] - 'a';
    count[index]++;
}

```

$s[] = \{ b, c, d, a, d, a \}$



// iterate on count[] and print

```

int k=0;
for (int i=0; i<26; i++) {

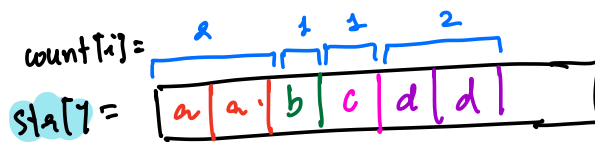
```

// count[i] = freq. of char with ASCII value (i+97)

```

    for (j=0; j<count[i]; j++) {
        str[k] = i + 'a';
        k++;
    }
}

```



This approach is called Count Sort

↓  
Advanced Batch Sorting class.

| i  | j          | iterations |
|----|------------|------------|
| 0  | [0, c[0]]  | count(a)   |
| 1  | [0, c[1]]  | count(b)   |
| 2  | [0, c[2]]  | count(c)   |
| ⋮  | ⋮          | ⋮          |
| 25 | [0, c[25]] | count('z') |

$T.C = O(N)$

Space complexity  $\Rightarrow O(1)$

$N \rightarrow$  size of i/p array.

Break  $\rightarrow$  10:35 PM

```

for (i=0; i<N; i++) {
    // index of s[i] in count[]
    int index = s[i] - 'a';
    count[index]++;
}

```

// iterate on count[] and print

```

int k=0; char str = new char[N];

```

```

for (int i=0; i<26; i++) {
    // count[i] = freq. of char with ASCII value (i+97)
    for (j=0; j<count[i]; j++) {
        str[k] = i + 'a';
        k++;
    }
}

```

$s[] = \{a, b, a, c, d\}$

count = {<sup>0</sup>2, <sup>1</sup>1, <sup>2</sup>1, <sup>3</sup>1, <sup>4</sup>0, <sup>5</sup>0, <sup>6</sup>1}

| i | count[i] | str[k].     |
|---|----------|-------------|
| 0 | 2        | aa          |
| 1 | 1        | aab         |
| 2 | 1        | aabc        |
| 3 | 1        | <u>abcd</u> |

\* Substring?  $\rightarrow$  like a subarray

$\rightarrow$  continuous part of a string

Q Given a string,  $s$ .

Given left and right indices.  
( $l$ ) ( $r$ )

Reverse string from index  $l$  to  $r$

e.g.  $\{ a, b, d, e, a, g, f \}$   
 $\{ a, b, g, a, e, d, f \}$

$\rightarrow$  string palindrome or not.

,  $l=2, r=5$

Todo

reverse( $s, l, r$ ) {

```
while ( $l < r$ ) {  
    swap  $s[l]$  and  $s[r]$   
     $l++$   
     $r--$   
}
```

Ques 5 Given a string. Return the length of longest palindromic substring

Amazon,  
Directi,  
Ola,  
Meta,  
Apple

Palindrome? → MADAM  
→ ABBA  
→ NAMAN  
→ RADAR  
→ MALAYALAM

e.g. a b a c a b → Ans = 5

→ single chars are also palindrome  
→ Is "a b" palindrome? → No

e.g. a b c d b → Ans = 1  
↳ Edge case

Idea 1:- 2 pointer Approach

a b a c a b → This type of 2 pointer won't work  
↑ ↑

Idea 2:-

a b a c a b → odd length substring

a b c d c b a

i 0 1 2 3 4 5 6 j 7  
x a b c | c b a y  
↳ center =

$$j - i - 1 = 7 - 0 - 1 = 6$$

$$\text{length} = j - i - 1$$

$$(a|b) = b - a - 1$$

0 1 2 3 4 5 6  
a b c d c b a → odd  
i, j  
 $s[i] == s[j]$

0 1 2 3 4 5  
a b c | c b a → even  
i j

```
int getPalLength (s, i, j) {
    while (i >= 0 && j < N && s[i] == s[j]) {
        i--;
        j++;
    }
    return j - i - 1;
}
```

$n$  chars are centers.

e.g. a | b | a | c | a | b  
↑ ↑ ↑ ↑ ↑ ↑

↑ → centers for odd length palindromic substrings →  $n$

| → centers for even length palindromic substring →  $n-1$

total =  $2n-1$  palindromic centers.

calls →  
 getPalLength (s, 0, 0) →  $i=0$  → 'a' as center  
 getPalLength (s, 0, 1) →  $i=0$  → 'a|b' as center  
 getPalLength (s, 1, 1) →  $i=1$  → 'b' as center  
 getPalLength (s, 1, 2) →  $i=1$  → 'b|a' as center  
 ...  
 ...  
 ...

→ For every character →  $(i, i)$  as center (for a char as center) → odd  
 →  $(i, i+1)$  as center (for a partition as center) → even

Pseudo code:-

```
int ans = 1 // min palindromic length
for (i = 0; i < N; i++) {
  // odd len Pal
  int len1 = getPalLength(s, i, i);
  // even len Pal
  int len2 = getPalLength(s, i, i+1);
  ans = max(ans, max(len1, len2));
}
return ans;
```

$$\begin{aligned} T.C &= (2N * N) \\ &= O(N^2) \\ S.C &= O(1) \end{aligned}$$

```
int getPalLength(s, i, j) {
  while (i >= 0 && j < N && s[i] == s[j]) {
    i--;
    j++;
  }
  return j - i - 1;
}
```

$$\rightarrow T.C = ? = O(N)$$

$(s, i, i+1)$

$i = N-1 \rightarrow (s, N-1, N) \rightarrow$  Already handled

When we learn Dynamic Programming.

$T.C = O(N^2)$   
 $S.C = O(N^2)$  } → Same Question

→ Directly →  $O(N)$  approach

↓  
Manacher's Algorithm

↓  
Feel free to explore. → Advance Batch.

